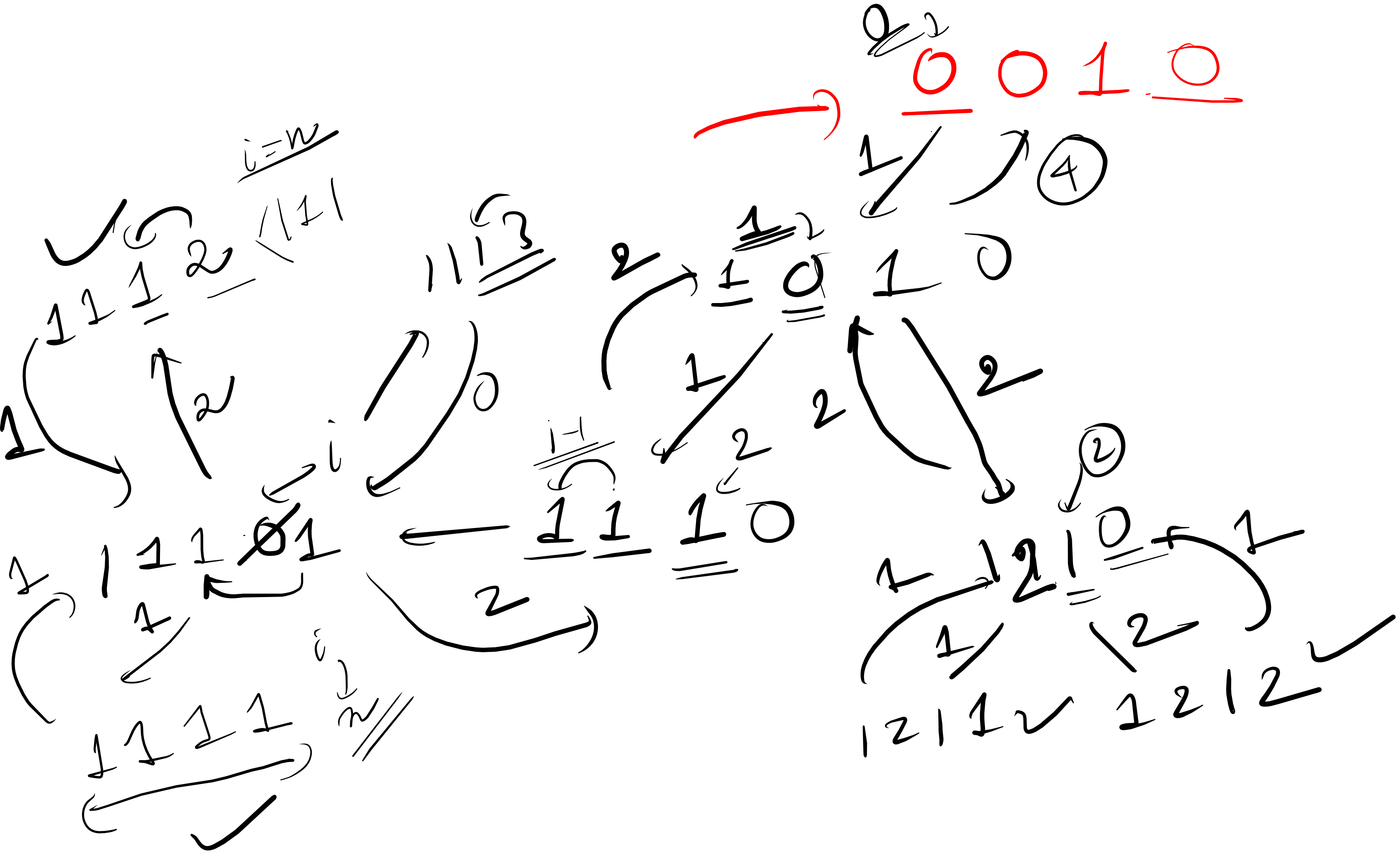
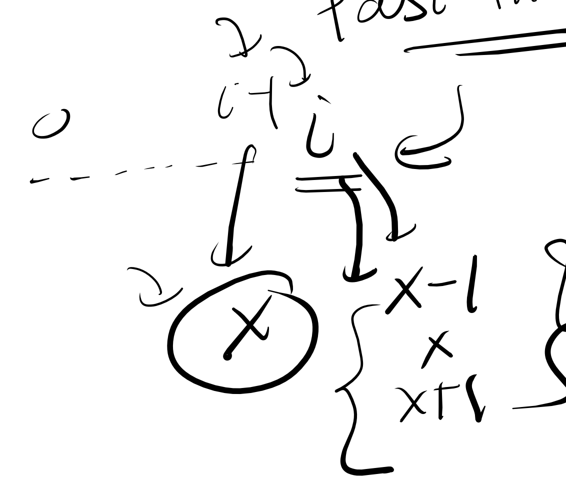
$n \leftarrow 4 \quad 3 \rightarrow m$

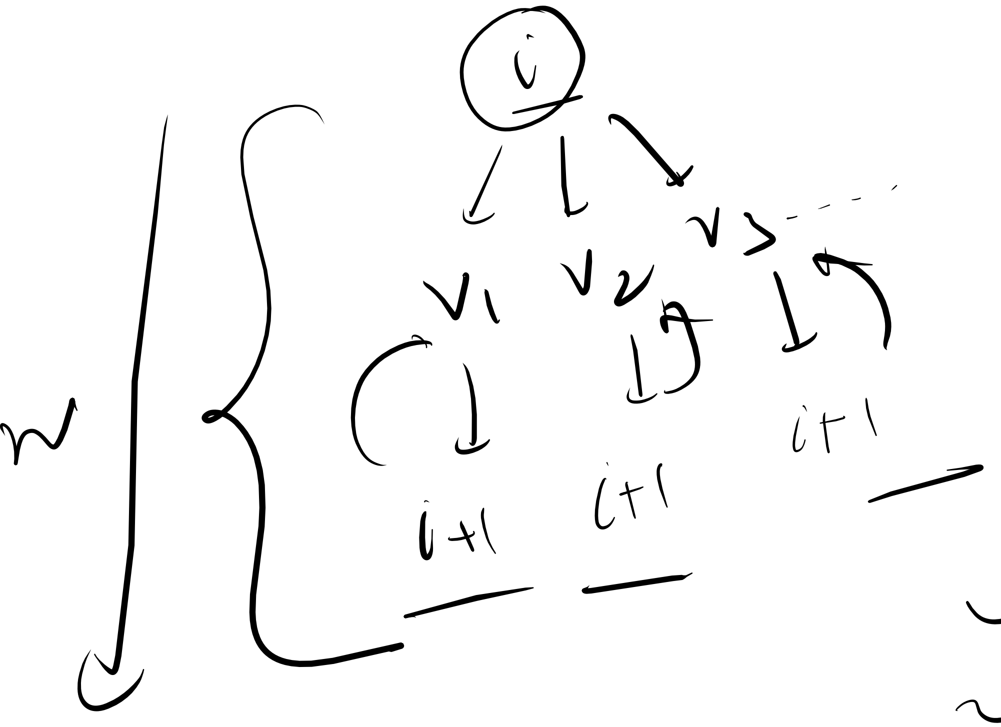
$e_i \to <= 3$



past index



$|e_{i-1} - e_i| <= 1$

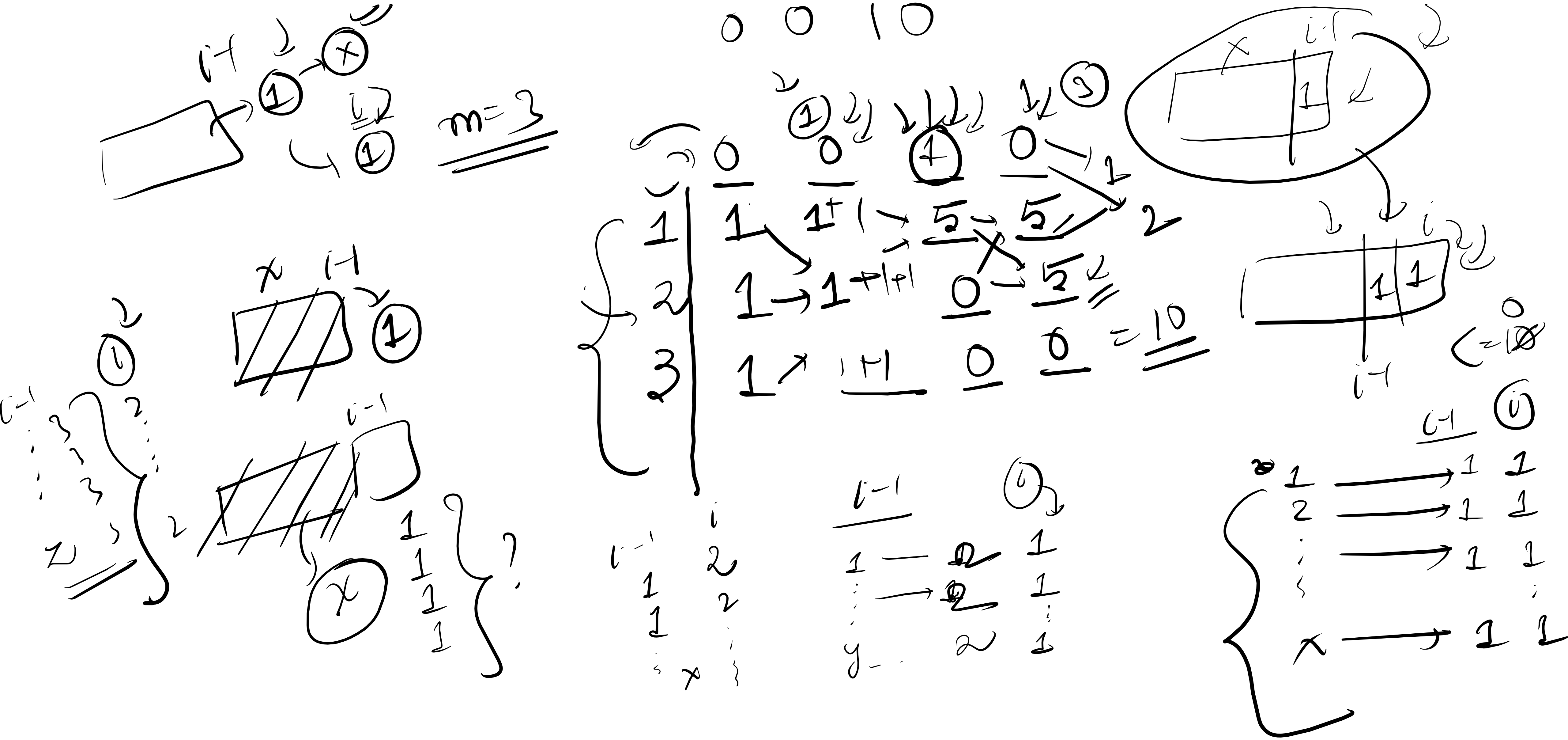


i = 0

i-1 ⊗

$O(3^n)$

```
f(i) {
    if(i == n)
        return 1;
    soln = 0;
    for(num=1; num<=m;
            num++) {
        if(i-1<0 ||
           abs(num - arr[i-1]) <= 1) {
            arr[i]=num;
            soln += f(i+1);
        }
    }
    return soln;
}
```


m=3
0 0 1 0
=10

B B ? ? 13

string

"ABCD"

A ? ? ? B C B ? ?

Upper case chars.

26

A

:

Z

i-1 i

i

{fixed}

{fixed + not} no. of string that is formable such that no two characters are same

counting

i j

i_2 j_2

soln?

$n \rightarrow O(n^2)$

$i \rightarrow K$

1 tap

DP

all vals of 'i'

INT_MAX

j

i-K-1

min taps

j

i-K

min

i

j

i+K

1 tap

dp[n]

n

n-1

0 . . .

i-K . . . i+K

$dp[j] \rightarrow \min(dp[j],\ 1 + dp[i-K-1])$ j

0 0 1 0

1

1 1 1 1

1 1 1 2

1 2 1 1

1 2 1 2

2 1 1 1

2 1 1 2

2 2 1 1

2 2 1 2

3 2 1 1

3 2 1 2

$k = 3$

abcabab

p

aba

p

t

??

$k-1$

$i \to n-1$

3

k

3 → t

$O(nk)$!

$i \to p$

k

$i \ldots i+k-1 \to t$!

$i = 4$

$n = 7$

$n-k$

$i \to 0 \ldots n-k$

i

$\hookrightarrow n-k-1$

i

$0 \ldots$

$n-k$

$n-k-1$

$+ (k-1)$

$n-\cancel{k}-1 + (\cancel{k}-1)$

$n-2$

Z → algo ?

index ← i

$s[i]$

prefix of s

for all index value i of string s (length n)

$\{ s[i] \ldots\ldots s[j] \}$

length

prefix of string s.

abcd

a
ab → 0
abc
abcd

1 . . . 2

s → ababcba

0 1 2 3 4 5

z[i] = 0 0 2 0 0 0 1 → z array

n

s[i] → longest substring prefix of 's' length

0 1
1
a

6 7
1
a

b . . . .
a

a b . . . . . .
0

$s = ababa$

$z \to 0\ 0\ 3\ 0\ 1$

$\frac{0}{a}\ \frac{1}{b}$

$0\ 1\ 2\ |\ 2\ 3\ 4\ 5$

$a\ b\ a\ \ a\ b\ a$

$\frac{0}{a}\ \frac{3}{b}$

$\frac{0}{a}\ \frac{4}{a}$

```
vector<int> z_function(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for(int i = 1; i < n; i++) {
        if(i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while(i + z[i] < n && s[z[i]] == s[i +
z[i]]) {
            z[i]++;
        }
        if(i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

```cpp
vector<int> z_function(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for(int i = 1; i < n; i++) {
        if(i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while(i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if(i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

i=1 i=2

abcabcab 8

z: 00000000

5

i=3

l=3

r=8

bcab

a

0

abcabcab

i+z[i]

i+z[i]

e

O(n)

O(n)

i=4 → 0

i=5 → 0

i=6 = 2

i=7 = 0

t

p abc

abcbcabc

0 5

z function

combined

abc$abcbcabc

3 0 0 0 0 3 0 0

i

2

$z[i] = \text{length}(t)$

find no. of times each prefix occurs within a string?

abaab

prefix

a → 2   1

2   1

1

1

1

prefix/suffix arr

1 . . i . . n

+1 → final no.

$z[i] = k$

1 → +1

k → +1

abcab

0 0 0 2 0

$z[i] = 2$?

ith → 2 → prefix

1?

1 → 2 prefix.

1 prefix

2 prefix

abcabacba

palindrome

abcaba ? ?

KMP algo

Manacher

abacba $ abcaba

rev

2 0 0 3 0 0

len - max(z[i])

??

prefix

i

n-1

?

ababa$ababa

5 0 3 0 1

?

r - s == 0

(L - z[i])

i -> z[i]

i+z[i] == n ?

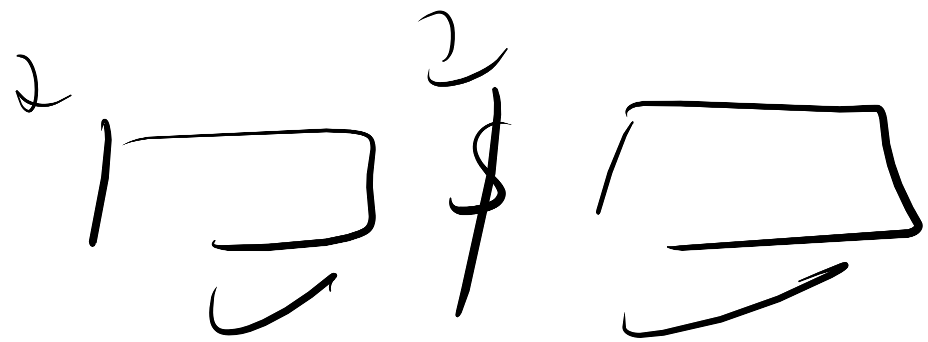
i

suffix

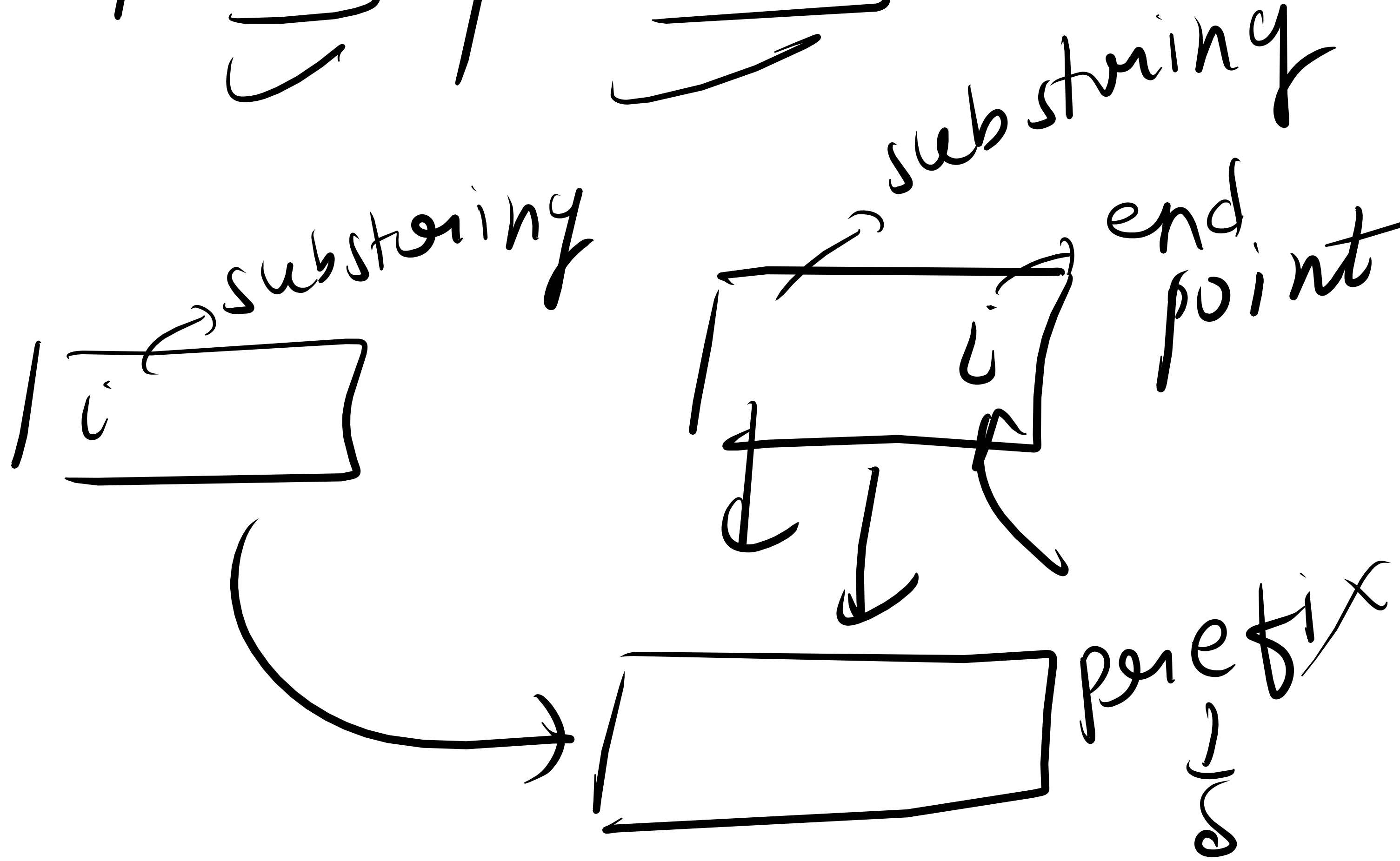
palindrom

Kmp → O(m+n) S_L

(2)



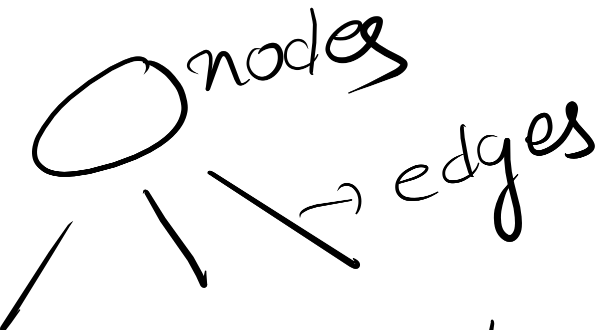
code



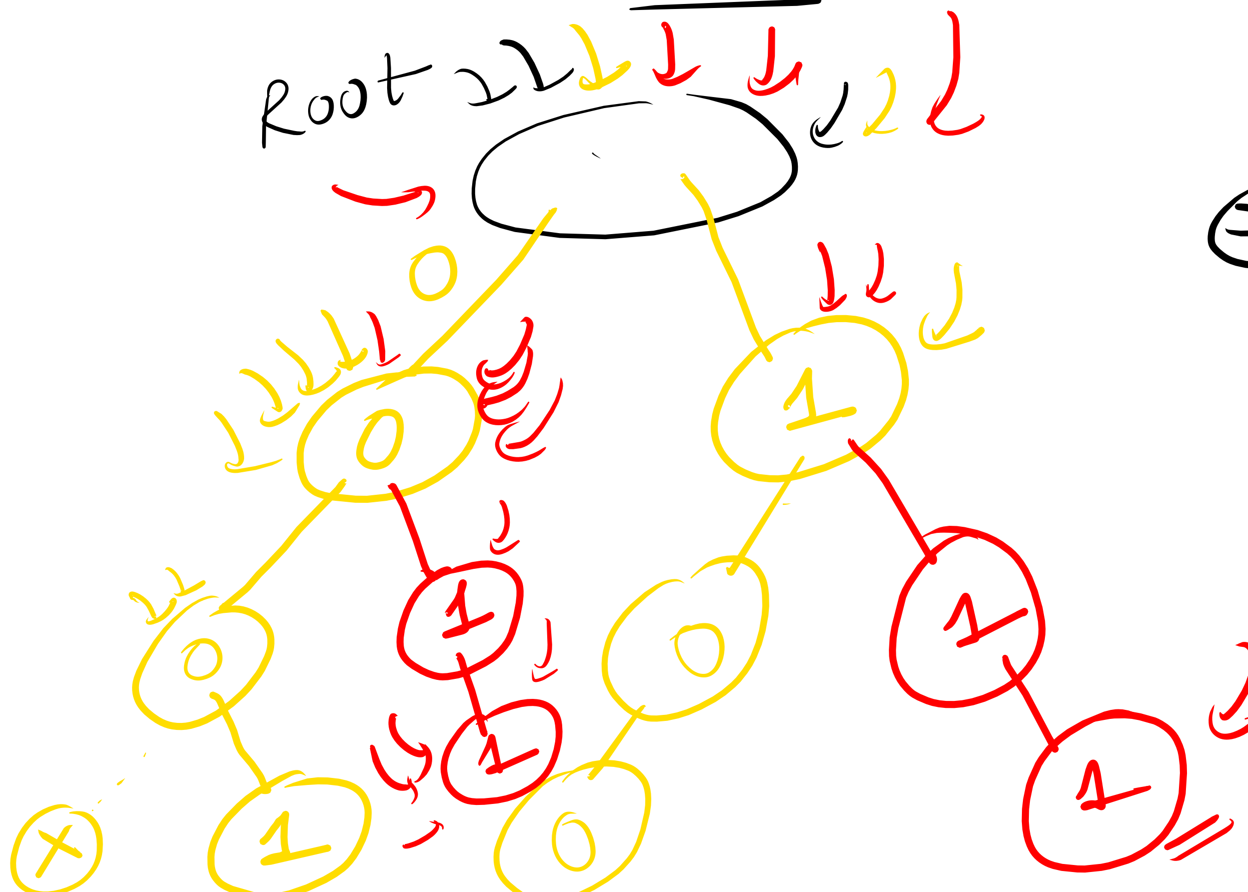
0/1 ← Trie → BIT

↓

tree



1/0



MSB → LSB

nums

1	→	0	0	1
4	→	1	0	0
3	→	0	1	1
7	→	1	1	1
		2	1	0

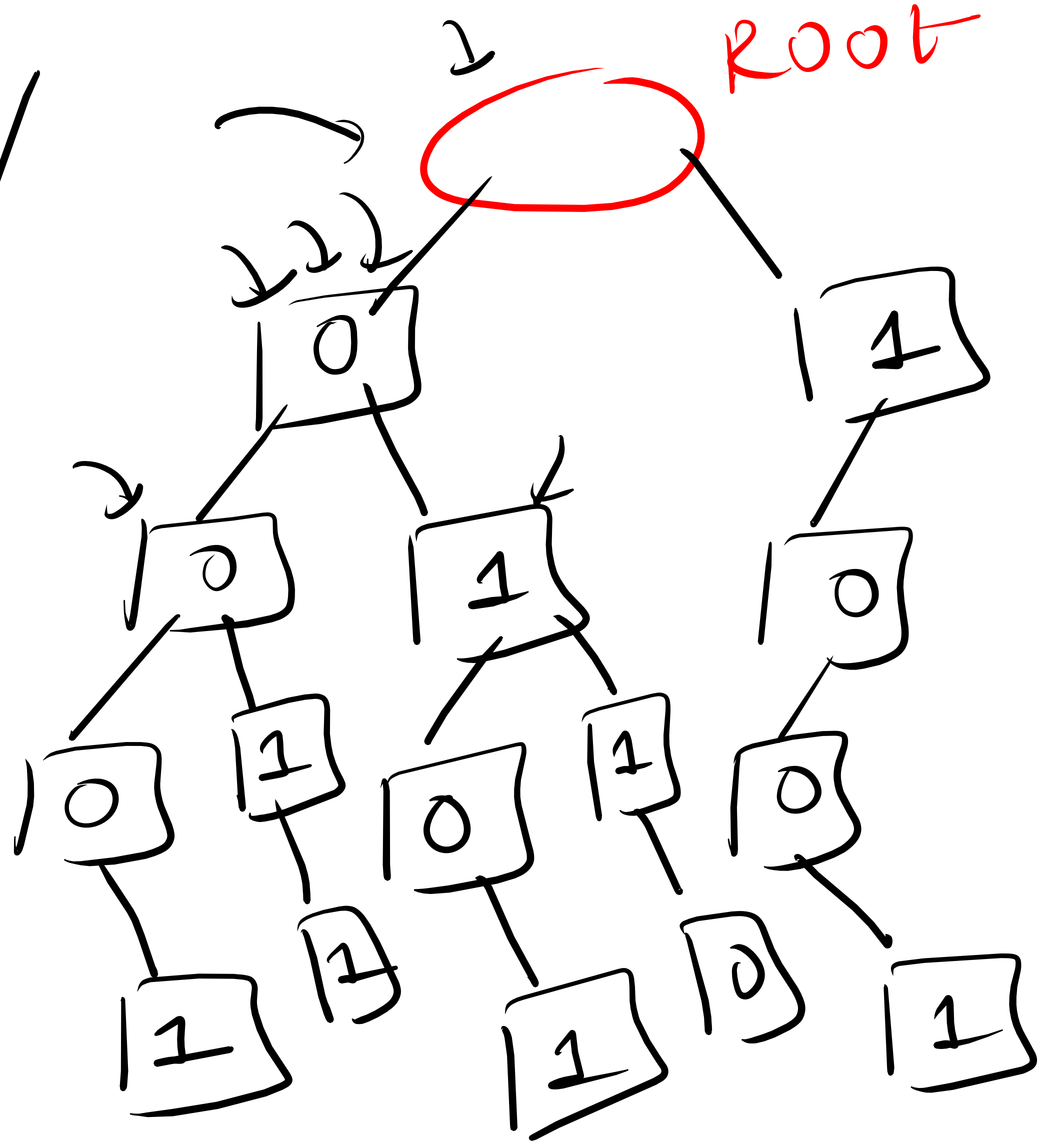
num → int

$\log_2(num)$

$\log_2(num)$

depth

Trie



1 5 9 3 6

1 → 0001

5 → 0101

9 → 1001

3 → 0011

6 → 0110

Node {

data

Node* arr[2]

};

new node

arr[0]

[1]

arr[0] → [0]th bit child

Null

arr[1] → Bit 1 child

map<char, Node*>;

```
class Node {

    int bit;

    Node* arr[2];

    Node(int bit) {
        this->bit = bit;
        arr[0] = NULL;
        arr[1] = NULL;
    }
};
```

{bit → 0/1}
bool → 0/1
true & false

```
insert(Node* root,
       int num) {

}
```

num → int

31 ... 0

problem

nums → int

$0 <= num <= 10^9$

MSB → 1 → 0

MSB

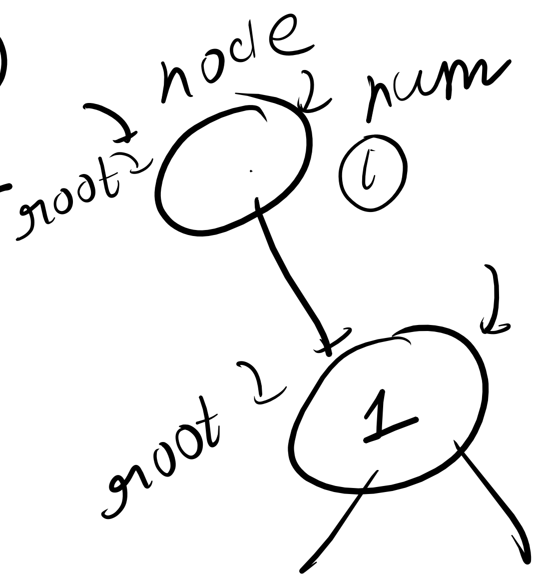
1

25

...

31

108 ← 4 → 2 ...




```
void insert(Node* root, int num){

  for(i=31; i>=0; i--){
    if(num&(1<<i)){
      if(root->arr[1]
            == NULL){
        root->arr[1]
            = new Node(1);
      }
      root = root->arr[1];
    }
```

nums

n

$n * \log_2(\text{MaxElement});$

$1 \ldots 10^9 \rightarrow \underline{100} / \underline{1000}$

2

max

```
else {
    if ( root->arr[0] == NULL){
        root->arr[0] = new
                       node(0);
    }
    root = root->arr[0];
}
}
```

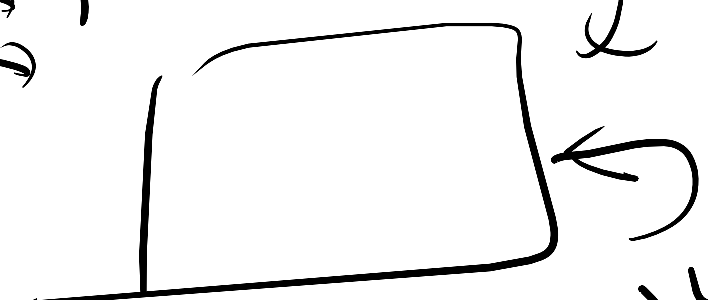
nums → max MSB

1000 → $\log_2(1000) + 1$

$O(n * \log_2(\text{Max}))$

Trie

n element



$\log_2(\text{Max})$

2

nums → 1 4 5 7 → int

Q → ~~num~~

x → int

T

BIT Trie

($\log_2$(Max))

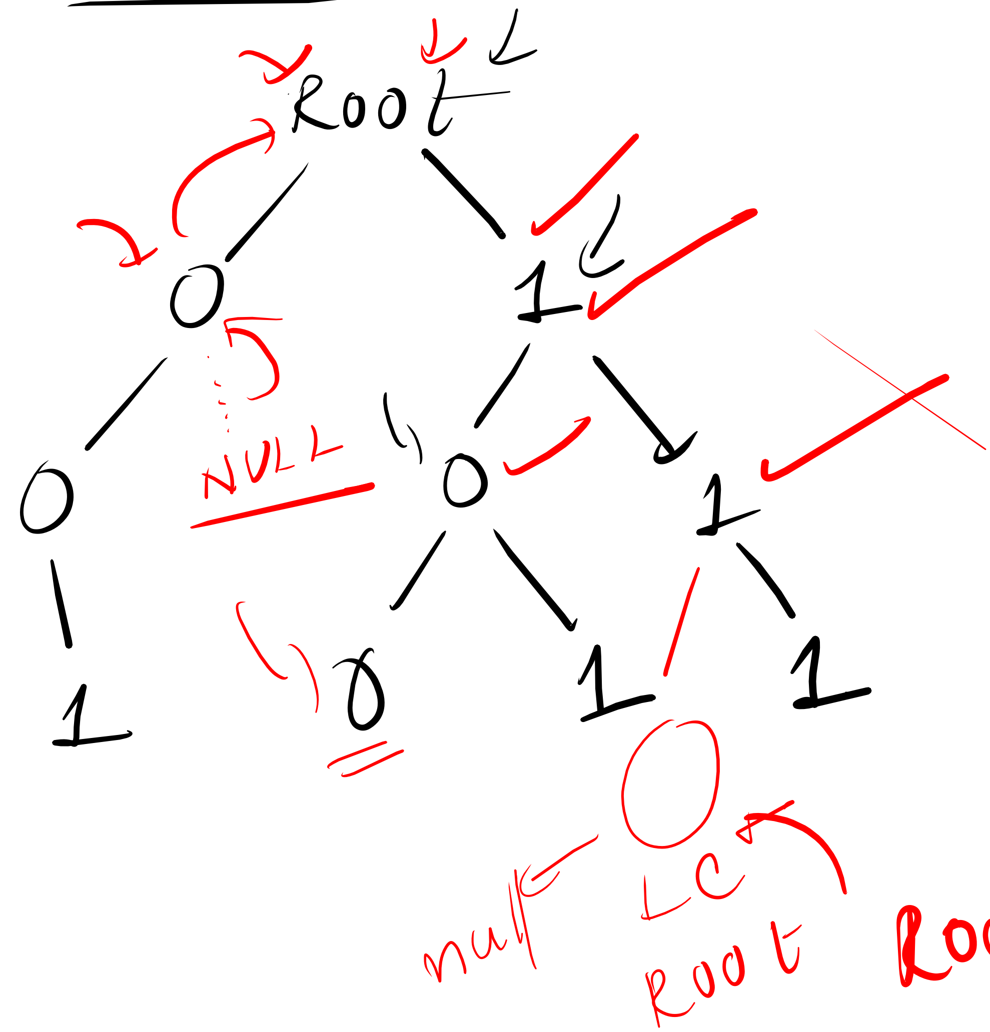
Q → M → n ops

X → arr

LSB

$1 \rightarrow 001$
$4 \rightarrow 100$
$5 \rightarrow 101$
$7 \rightarrow 111$

1 4 5 7



Query

$x = 4$

100

True

$x = 2$

010

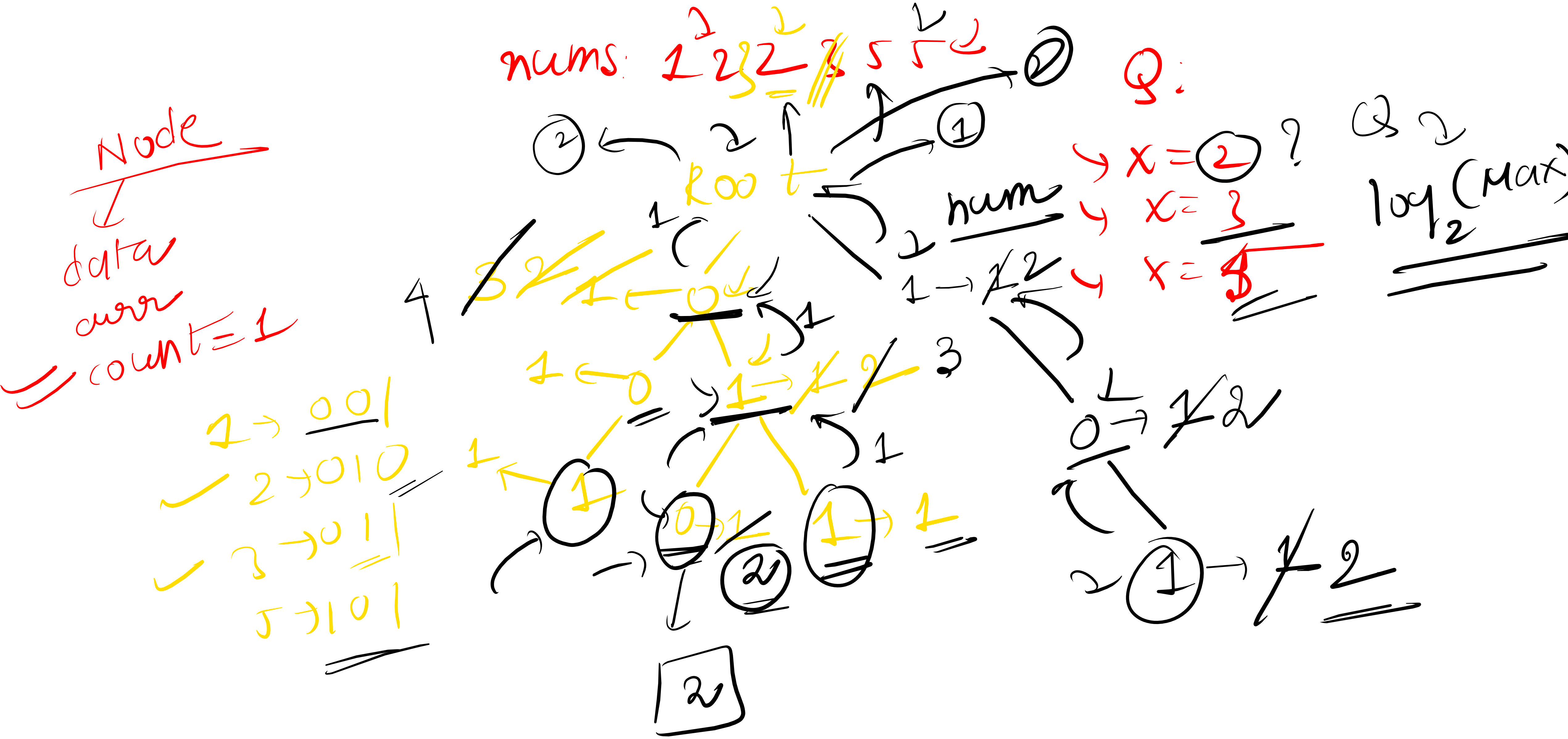
Root = NULL

false

$x = 6$

110

false



nums: 1 2 3 2 5 5
Node
data
curr
count=1
root
num
Q:
x = 2 ?
x = 3
x = 5
log2(Max)
1 → 001
2 → 010
3 → 011
5 → 101

element ans = ~~0~~ 5

~~5~~ 6 → ans

nums → 1 4 5 7

max xor

nC_2

n^2

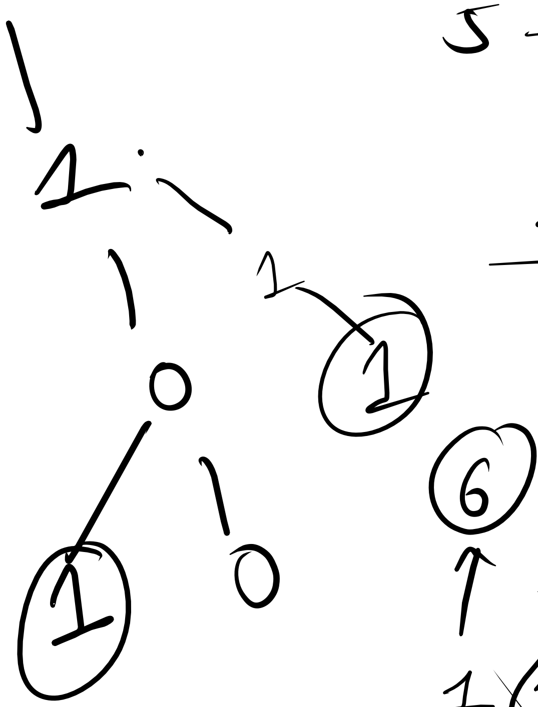
$x \wedge y$

figure out

4 → 100

5 → 101

root



1 0 1

1 0 0

1 1 1

1 1 0 0 0 ~~0~~ 1

string
Trie

subarray → longest
XOR
of
elements

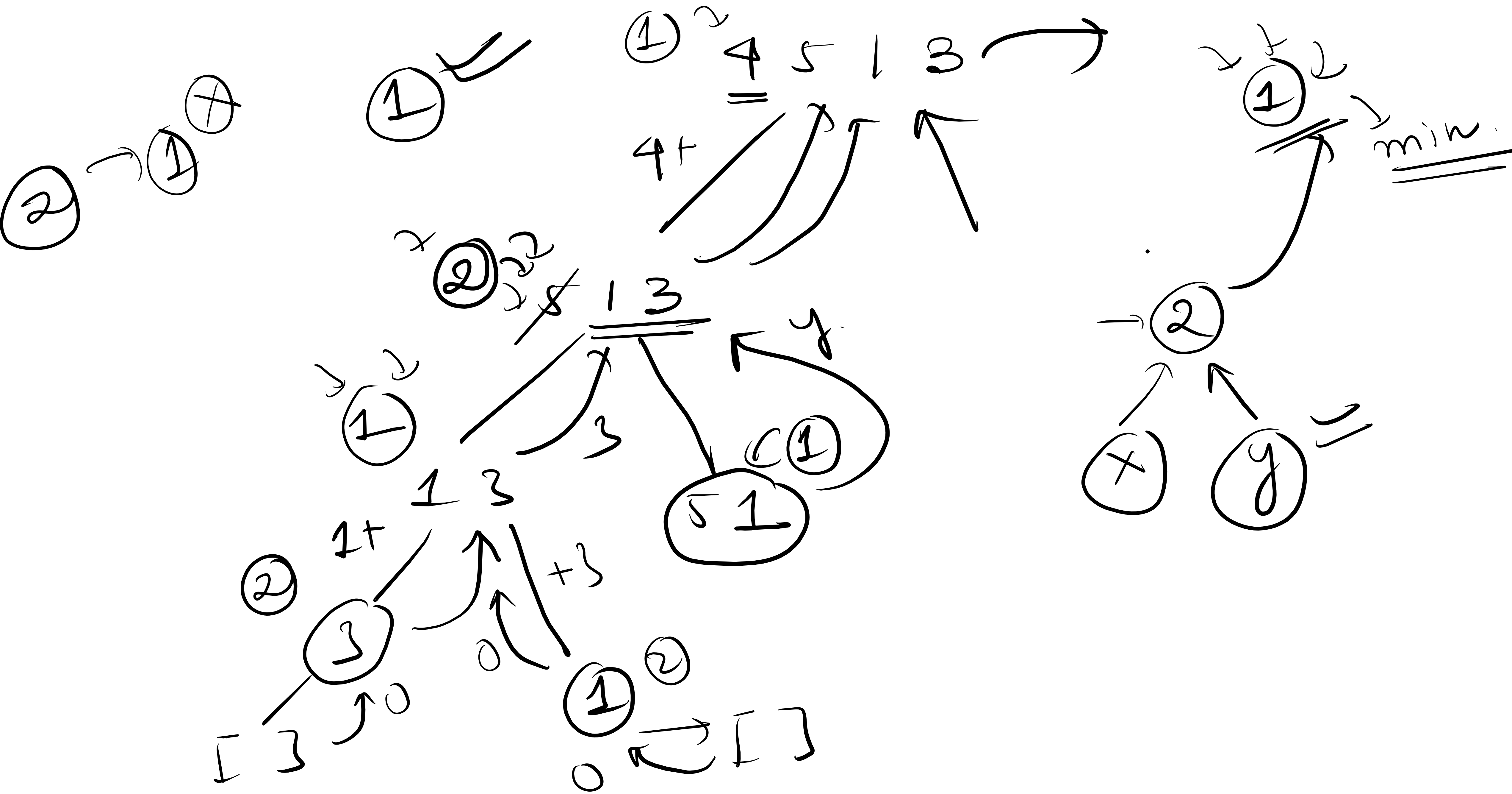
BIT
string
algo

Z

KMP / manacher

substring

$$O\left(n * \log_2(\max)\right)$$



4 5 1 3
4+
5 1 3
1 3
1+
+3
min.
x
y

$\cancel{[st \; \cdots \; en]}$

2D $\; f(st, en)$ { $\quad$ turn $\quad$ st, en ?

if (st > en)
 return 0;

{ moves of player 1 }

$st \cdots en$

i $\quad$ j

$s_1 = arr[st] +$

st+1 .. en

p2

(st+1 ... en)

min($\quad f(st+2, en),$

p2

$f(st+1, en-1)$);

$s_2 = arr[en] +$

st . . en-1

min($f(st+1, en-1)$,

$f(st, en-2))$;

]+

P1 → s_1 s_2

return max(s_1, s_2);

}

4+1

5+

4 5 1

P2 2 P1

$i+2<n$

$j-2 / j-1 >= 0$

$\rightarrow n*n$

8

2

4 5 1 3

(0 ... 3)

	0	1	2	3
0	4	5	5	8
1	0	5	5	6
2	0	0	1	3
3	0	0	0	3

2

8 ✓

$S_1 = 7$ min $\rightarrow$ 1

min(

$dp[i][j] =$

$arr[i] + \min(dp[i+2][j], dp[i+1][j-1])$

$arr[j] + \min(dp[i+1][j-1], dp[i][j-2])$,

$dp[0][3]$ —

8 ✓

$S_2 = 3 + dp[0][1]$

$3 + 5$ $dp[1][2]$

$= 8$

)

i K (K+1 . . j)

dp[i][j] = false

f(i, j) → wordDict?

i - - K

wordDict?

i . - - j

WD

```
for(K = i; K <= j; K++) {
    if(WD.count(s.substr(i, K)) == 1) {
        if(f(K+1, j) == true)
            dp[i][j] == true;
            return true;
    }
}
```

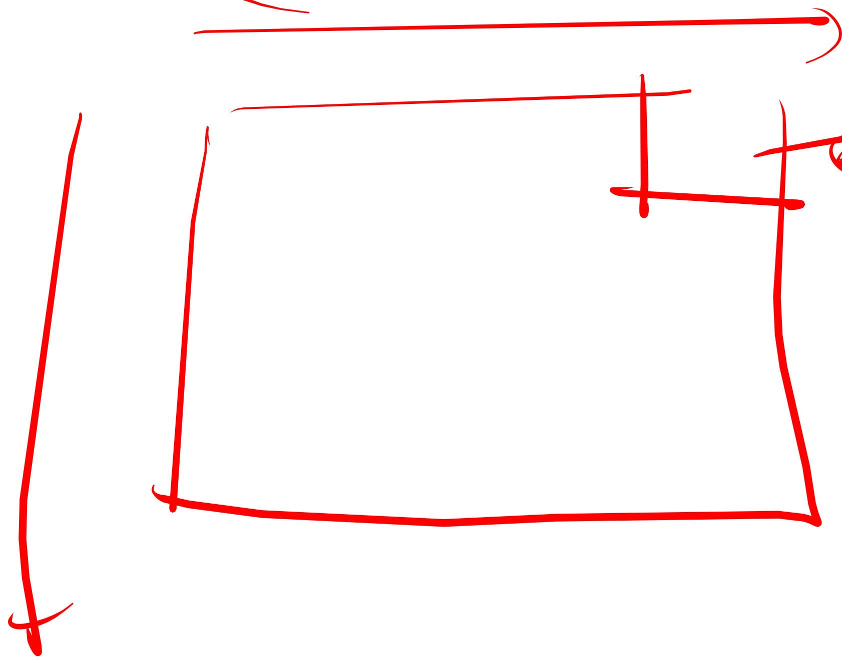
i+1, j

i . . . K

(K+1 . . . j)

i j

(i . . . K) (K+1 . . . j)



→ true → return

false

i → j → True/false

$n_1 = 4$ 0 0 1 0 → 10

$n_2 = 4$ 0 1 0 1 → 101

$\cancel{0}$ 1 1 1

1
1
1

0 0 | 1 0 0

1
0
0

res → ~~0 0 0~~ 5

10077 ?

$c = 1$

$$\begin{array}{r} 1\;1\;\;1 \\ 0\;0\;9\;9 \\ +\;\;9\;9\;7\;8 \\ \hline 1\;0\;0\;7\;7 \end{array}$$

$d_1 + d_2 = 17$

$17 \% 10 = 7$

$17 / 10 = 1$

$9 + 1 = 10 \% 10 = 0$

$9 + 7 + 1 = 17$

$10 / 10 = 1$

~~$9 + 10 =$~~

$9 + 1 = 10 \% 10 = 0$

$17 \% 10 = 7$ ✓

$17 / 10 = 1$

15 10 5 11 14

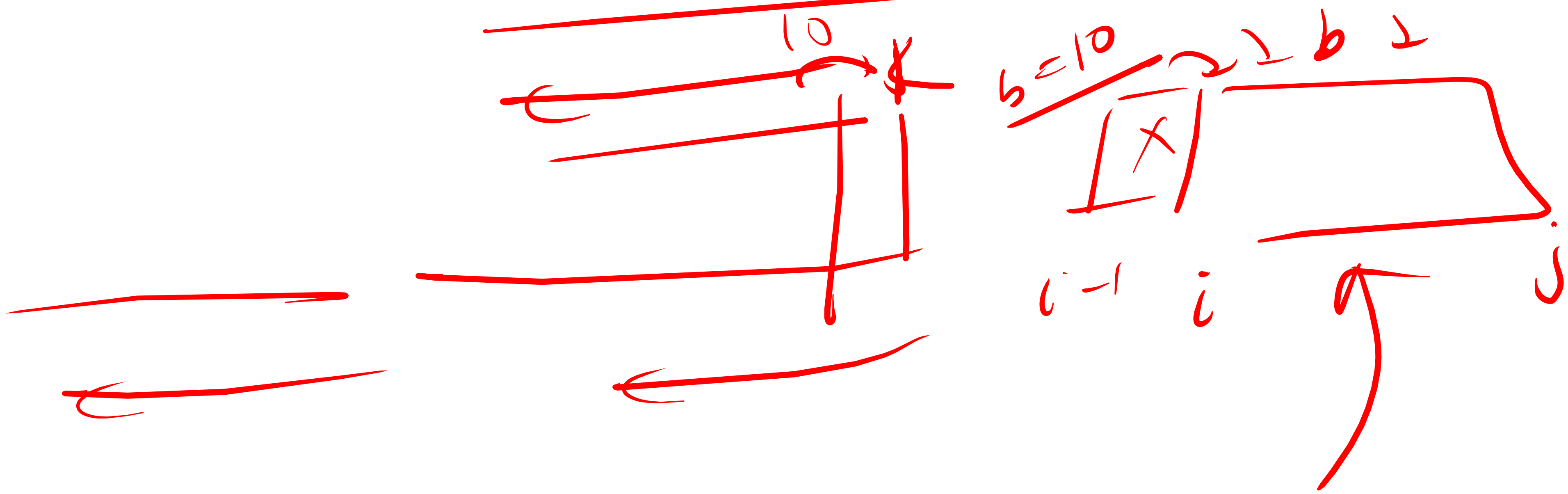
6 7 8 6 2 4 3

0 7 5 9 6 2

6 8 0 2 8 1 1

$a_2[i] > a_1[i]$

$b = 9$ 10 9



$b=9$

d_1

d_2

$\cancel{0}\ \cancel{1}\ \cancel{5}\ \cancel{2}\ 12$

$0\ 5\ 9$

$0\ 9\ 3\ 2$

$b=\cancel{10}\ 9$

size is fixed

arr → n

non resizable.

1000

arr

100

short

10

arr

10000

arr → n

arr < 10

→ 10

→ 50

arr → 60

v[idx].

STL → container

arrays

vector

array

dynamic arr.

grow

shrink

indexing

vector

array

result

2D arr

	0	1
0 →	1	2
1 →	3	4
2 →	5	6

← 2D arr

arr[1][1] → 4 ✓

same

2D arrays ?

[7, 8, 9] → 1D array

indices: 0, 1, 2

[[1 2], [3 4], [5 6]] → 2D array

indices: 0, 1, 2

array of arrays

$2*2$ $2*2$

arr →

$$\begin{array}{c|cc} & 0 & 1 \\ \hline 0 & 0 & 0 \\ 1 & 0 & 0 \end{array}$$

```
int arr[ ] = new int[n];
```

m, n

```
int arr[ ][ ] = new int[n][m];
```

row by row

	j=0	j=1	j=2
i=0:	1	2	
i=1:	3	4	
i=2:			

Col wise printing

~~m → 10~~
n → rows
m → cols.

$$\begin{array}{c|ccc} & j=0 & j=1 & j=2 \\ i=0 & 1 & 2 & 3 \\ i=1 & 4 & 5 & 6 \end{array} \Rightarrow \begin{Bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{Bmatrix}_{\text{x}}$$

```
for(j=0; j<m; j++)
    for(i=0; i<n; i++)
        print(mat[i][j]);
```

$$
\begin{array}{c|cccc}
 & K & K & K & K \\
\hline
i \rightarrow & 1 & 2 & 3 & 4 \\
j \rightarrow & 5 & 6 & 7 & 8 \\
i\ j \rightarrow & 9 & 10 & 11 & 12 \\
j \rightarrow & 13 & 14 & 15 & 16
\end{array}
\quad \rightarrow \quad
\begin{array}{cccc}
13 & 14 & 15 & 16 \\
9 & 10 & 11 & 12 \\
5 & 6 & 7 & 8 \\
1 & 2 & 3 & 4
\end{array}
$$

$i \leftrightarrow j$

$K=0 \quad K=1 \quad K=2 \quad K=3$

```
i=0 / j=n-1
while(i<j){
    k -> 0 ... m-1
    swap(mat[i][k], mat[j][k]);
    i++;
    j--;
}
```